Lab 5 Practice: Random Memory Access

This practice does not need to be submitted.

Objective

This practice introduces the basic operations of random-access memory (RAM). You will design and verify a **RAM with one read port and one write port (1R1W)**.

The goal is to understand:

- Random memory read/write operations
- Write-first memory behavior
- Valid and invalid memory locations
- Address mapping from an external address to a memory index

Practice Task

Implement a memory array consisting of eight 4-bit entries.

The circuit supports:

- One write port
- One read port
- Write-first policy
- A valid-entry counter to track the number of valid memory entries

a. I/O list

Signal	I/O	Width	Description
clk	Input	1	Positive-edge triggered clock signal
rst	Input	1	Synchronous active-high reset. When rst == 1'b1, all memory entries are cleared to zero, all valid bits are cleared, cnt_valid is cleared to zero, and dout is cleared to zero on the next rising clock edge.
we	Input	1	Write enable. When we == 1'b1, din is written into the memory entry selected by the mapped write address on the rising clock edge.

w_addr	Input	7	Original write address, ranging from 0 to 127. Before accessing memory, it is converted into a 3-bit memory index using the specified address-mapping function.
din	Input	4	Data to be written into the selected memory entry.
re	Input	1	Read enable. When re == 1'b1, the selected memory entry is read on the rising clock edge.
r_addr	Input	7	Original read address, ranging from 0 to 127. Before accessing memory, it is converted into a 3-bit memory index using the specified address-mapping function.
cnt_valid	Output	4	Number of valid memory entries. The value ranges from 0 to 8. It increases by one only when data is written into an entry that was previously invalid. Overwriting an already valid entry does not change this value.
dout	Output	4	Registered read-data output. When re == 1'b1, it outputs the data stored in the selected memory entry. When re == 1'b0, it outputs zero.

b. Specification:

- The memory of eight 4-bit entries can be modeled as
  ```
  logic [3:0] memory [0:7];
  ```
  in SystemVerilog.
- Upon reset, all memory contents are cleared to zero. You may use a for-loop to clear all memory entries.
- Each memory entry has a valid bit.
  - Upon reset, all valid bits are cleared (invalid).
  - An entry becomes valid after it is written for the first time.
- Address Mapping:

  The memory contains only eight entries, so a 3-bit memory index (0 to 7) is used to select one of the eight entries. However, the original addresses w_addr and r_addr are 7-bit values ranging from 0 to 127.

  Before accessing the memory, the circuit must convert each 7-bit input address into a 3-bit memory index. In this lab, the conversion is performed using the following XOR-folding operation:

mapped_addr = addr[2:0] ^ addr[6:4] ^ addr[3:1];

The bitwise XOR produces a 3-bit result:

7-bit input address → address mapping → 3-bit memory index

For example:

addr = 7'b1011010 (5A)

addr[2:0] = 3'b010

addr[6:4] = 3'b101

addr[3:1] = 3'b101

mapped_addr = 3'b010 ^ 3'b101 ^ 3'b101 = 3'b010

Therefore, this input address accesses: memory[3'b010] = memory[2]

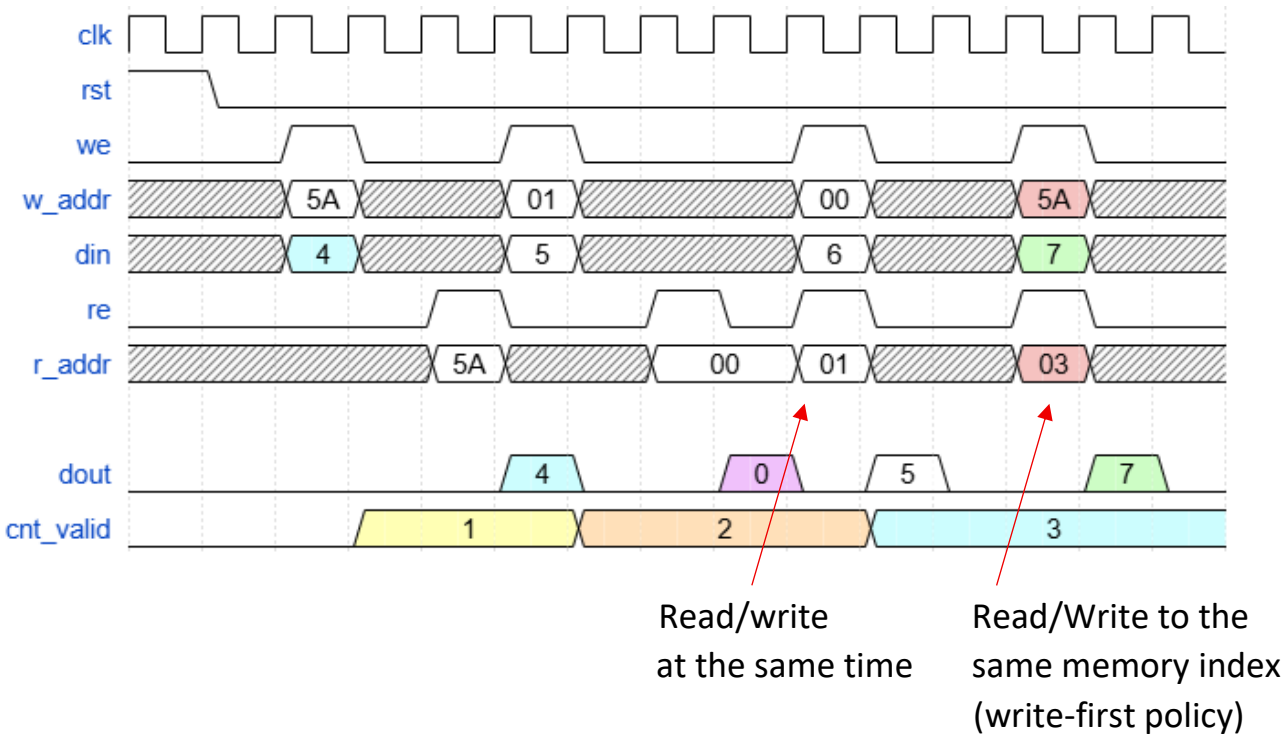
So, w_addr and r_addr are not used directly as memory-array indices. The mapped 3-bit addresses are used to access the memory.

- Read operation:
  - Read operations are synchronous and occur on the positive edge of clk.
  - When re == 1'b1, r_addr is converted into a 3-bit memory index.
  - If the selected memory entry is valid, dout is updated with the data stored in that entry.
  - If the selected memory entry is invalid, dout is set to 4'b0000.
  - When re == 1'b0, dout is set to 4'b0000 on the next rising clock edge.
- Write operation:
  - Write operations are synchronous and occur on the positive edge of clk.
  - When we == 1'b1, w_addr is converted into a 3-bit memory index, and the value of din is written into the memory entry selected by the mapped write address.
  - After a memory entry is written, its valid bit is set to 1'b1.
  - If the entry was previously invalid, cnt_valid is increased by one.
  - If the entry was already valid, the stored data is overwritten, but cnt_valid remains unchanged.
  - **Write-First Policy**: When we == 1'b1 and re == 1'b1 in the same clock cycle, if the mapped write and read addresses are equal, dout must return the newly written value (din).

    Therefore, whether the read and write access the same entry is determined by comparing their mapped memory indices, rather than their original 7-bit addresses.
- Read out the values stored in the memory to verify correctness.

- Refer to the following timing diagram as an example.
  Note that the slashed regions in the diagram represent **don't-care values**, not **unknown values**, meaning the logic levels at these points are irrelevant to the behavior of the design.

c. Waveform:



Note: Although 5A and 03 are different original addresses, both are mapped to memory index 2, so the write-first policy should be applied.

d. Template

Use the following template as the basis for your design:

```
module lab5_practice (
  input logic clk,
  input logic rst,

  // write port
  input logic we,
  input logic [6:0] w_addr,
  input logic [3:0] din,

  // read port
  input logic re,
```

```
    input logic [6:0] r_addr,
    output logic [3:0] dout,

    // Status output
    output logic [3:0] cnt_valid
);
    // Add your design here.
    // You may change the data types of the I/O signals.
endmodule
```

Attention

- This practice prepares you for the next lab. Make sure you fully understand your design.
- Feel free to ask questions about the specification on the EECLASS forum.
- DO NOT copy-and-paste code segments from PDFs (hidden characters may cause hard-to-debug syntax errors).